ECS 409/609
Assignment 1
Sequential Construct-I
Integer Arithmetic and Logical Instructions

Kunwar Arpit Singh

Submission Deadline: August 28, 2025
For code repository of this assignment: **MIPS Assignments**

---

1. Problem 1

Problem statement

For a given arithmetic and geometric progression sequence:

- 1, 11, 21, 31, ... (A.P.)
- 4, 8, 16, ... (G.P.)

Write a MIPS assembly program to compute the eighth term (in A.P.) / fourth term (in G.P.) and sum of the first six terms (in A.P.) / first four numbers (in G.P.). The initial values are stored in registers or memory.

Part A: Arithmetic Progression

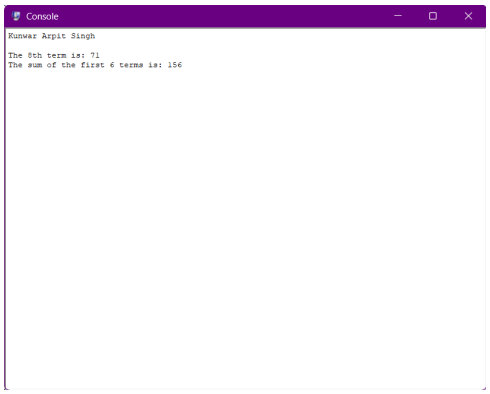
MIPS Assembly Source

```
.data
       myName: .asciiz "Kunwar␣Arpit␣Singh\n\n"
       term_msg: .asciiz "The␣8th␣term␣is:␣"
       sum_msg: .asciiz "\nThe␣sum␣of␣the␣first␣6␣terms␣is:␣"
       newline: .asciiz "\n"

.text
.globl main

main:
       li $v0, 4
       la $a0, myName
       syscall

       # $t0 = a
       li $t0, 1

       # $t1 = d
```

```
        li $t1, 10

        # $t2 = n
        li $t2, 8

        # Calculate the 8th term (n=8)

        addi $t3, $t2, -1

        mul $t4, $t3, $t1

        add $s0, $t0, $t4

        # Calculate the sum of the first 6 terms (n=6)

        li $t2, 6

        addi $t3, $t2, -1

        mul $t4, $t3, $t1

        mul $t5, $t0, 2

        add $t6, $t5, $t4

        li $t8, 2
        div $t2, $t8

        mflo $t7

        mul $s1, $t7, $t6

        li $v0, 4
        la $a0, term_msg
        syscall

        li $v0, 1
        move $a0, $s0
        syscall

        li $v0, 4
        la $a0, sum_msg
        syscall

        li $v0, 1
        move $a0, $s1
        syscall

        li $v0, 10
        syscall
```

Output Screenshots

Console output (A.P.)



Register values (A.P.)

Int Regs [16]

```
PC        = 4000ac
EPC       = 0
Cause     = 0
BadVAddr  = 0
Status    = 3000ff10

HI        = 0
LO        = 9c

R0  [r0] = 0
R1  [at] = 10010000
R2  [v0] = a
R3  [v1] = 0
R4  [a0] = 9c
R5  [a1] = 7fffef20
R6  [a2] = 7fffef30
R7  [a3] = 0
R8  [t0] = 1
R9  [t1] = a
R10 [t2] = 6
R11 [t3] = 5
R12 [t4] = 32
R13 [t5] = 2
R14 [t6] = 34
R15 [t7] = 3
R16 [s0] = 47
R17 [s1] = 9c
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0
R21 [s5] = 0
R22 [s6] = 0
R23 [s7] = 0
R24 [t8] = 2
R25 [t9] = 0
R26 [k0] = 0
R27 [k1] = 0
R28 [gp] = 10008000
R29 [sp] = 7fffef1c
R30 [s8] = 0
R31 [ra] = 400018
```

Part B: Geometric Progression

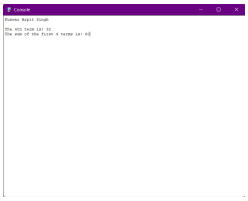
MIPS Assembly Source

```
.data
        myName: .asciiz "Kunwar Arpit Singh\n\n"
        term_msg: .asciiz "The 4th term is: "
        sum_msg: .asciiz "\nThe sum of the first 4 terms is: "

.text
.globl main

main:
        li $v0, 4
        la $a0, myName
        syscall

        # $t0 = a
        li $t0, 4

        # $t1 = r
        li $t1, 2

        # $t2 = n
        li $t2, 4

        mul $t3, $t1, $t1
        mul $t3, $t3, $t1

        mul $s2, $t0, $t3

        # Calculate the sum of the first 4 terms (n=4)

        mul $t4, $t3, $t1
        addi $t5, $t4, -1
        mul $t6, $t0, $t5
        addi $t7, $t1, -1

        div $t6, $t7
        mflo $s3

        li $v0, 4
        la $a0, term_msg
        syscall

        li $v0, 1
        move $a0, $s2
        syscall

        li $v0, 4
        la $a0, sum_msg
        syscall

```

```
50          li $v0, 1
51          move $a0, $s3
52          syscall
53
54          li $v0, 10
55      syscall
```

Output Screenshots

Console output (G.P.)



Register values (G.P.)

Int Regs [16]

```
PC        = 40009c
EPC       = 0
Cause     = 0
BadVAddr  = 0
Status    = 3000ff10

HI        = 0
LO        = 3c

R0  [r0] = 0
R1  [at] = 10010000
R2  [v0] = a
R3  [v1] = 0
R4  [a0] = 3c
R5  [a1] = 7fffef20
R6  [a2] = 7fffef30
R7  [a3] = 0
R8  [t0] = 4
R9  [t1] = 2
R10 [t2] = 4
R11 [t3] = 8
R12 [t4] = 10
R13 [t5] = f
R14 [t6] = 3c
R15 [t7] = 1
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 20
R19 [s3] = 3c
R20 [s4] = 0
R21 [s5] = 0
R22 [s6] = 0
R23 [s7] = 0
R24 [t8] = 0
R25 [t9] = 0
R26 [k0] = 0
R27 [k1] = 0
R28 [gp] = 10008000
R29 [sp] = 7fffef1c
R30 [s8] = 0
R31 [ra] = 400018
```

2. Problem 2

Problem statement

Write a MIPS assembly program to implement a half adder and a half subtractor using logical instructions.

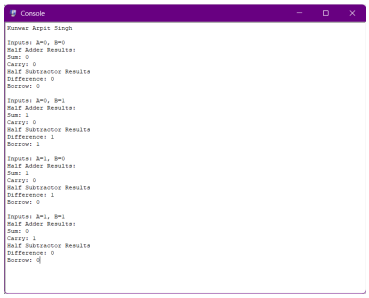
MIPS Assembly Source

```
.data
        myName: .asciiz "Kunwar␣Arpit␣Singh\n"
        inputs_00: .asciiz "\nInputs:␣A=0,␣B=0"
        inputs_01: .asciiz "\n\nInputs:␣A=0,␣B=1"
        inputs_10: .asciiz "\n\nInputs:␣A=1,␣B=0"
        inputs_11: .asciiz "\n\nInputs:␣A=1,␣B=1"

        adder_title: .asciiz "\nHalf␣Adder␣Results:"
        sum_msg: .asciiz "\nSum:␣"
        carry_msg: .asciiz "\nCarry:␣"
        sub_title: .asciiz "\nHalf␣Subtractor␣Results"
        diff_msg: .asciiz "\nDifference:␣"
        borrow_msg: .asciiz "\nBorrow:␣"

.text
.globl main

main:
        # Print name
        li $v0, 4
        la $a0, myName
        syscall
        li $t0, 0
        li $t1, 0
        li $v0, 4
        la $a0, inputs_00
        syscall
        # Calculations
        xor $s0, $t0, $t1
        and $s1, $t0, $t1
        xor $s2, $t0, $t1
        not $t5, $t0
        and $s3, $t5, $t1
        jal PrintResults
        li $t0, 0
        li $t1, 1
        li $v0, 4
        la $a0, inputs_01
        syscall
        xor $s0, $t0, $t1
        and $s1, $t0, $t1
        xor $s2, $t0, $t1
```

```
        not $t5, $t0
        and $s3, $t5, $t1
        jal PrintResults
        li $t0, 1
        li $t1, 0
        li $v0, 4
        la $a0, inputs_10
        syscall
        xor $s0, $t0, $t1
        and $s1, $t0, $t1
        xor $s2, $t0, $t1
        not $t5, $t0
        and $s3, $t5, $t1
        jal PrintResults
        li $t0, 1
        li $t1, 1
        li $v0, 4
        la $a0, inputs_11
        syscall
        xor $s0, $t0, $t1
        and $s1, $t0, $t1
        xor $s2, $t0, $t1
        not $t5, $t0
        and $s3, $t5, $t1
        jal PrintResults
        li $v0, 10
        syscall
PrintResults:
        li $v0, 4; la $a0, adder_title; syscall
        li $v0, 4; la $a0, sum_msg; syscall
        li $v0, 1; move $a0, $s0; syscall
        li $v0, 4; la $a0, carry_msg; syscall
        li $v0, 1; move $a0, $s1; syscall

        li $v0, 4; la $a0, sub_title; syscall
        li $v0, 4; la $a0, diff_msg; syscall
        li $v0, 1; move $a0, $s2; syscall
        li $v0, 4; la $a0, borrow_msg; syscall
        li $v0, 1; move $a0, $s3; syscall
        jr $ra
```

Output Screenshots

Console output (Half Adder/Subtractor)



Register values (Half Adder/Subtractor)

Int Regs [16]

```
PC        = 400114
EPC       = 0
Cause     = 0
BadVAddr  = 0
Status    = 3000ff10

HI        = 0
LO        = 0

R0  [r0]  = 0
R1  [at]  = 10010000
R2  [v0]  = a
R3  [v1]  = 0
R4  [a0]  = 0
R5  [a1]  = 7fffef20
R6  [a2]  = 7fffef30
R7  [a3]  = 0
R8  [t0]  = 1
R9  [t1]  = 1
R10 [t2]  = 0
R11 [t3]  = 0
R12 [t4]  = 0
R13 [t5]  = fffffffe
R14 [t6]  = 0
R15 [t7]  = 0
R16 [s0]  = 0
R17 [s1]  = 1
R18 [s2]  = 0
R19 [s3]  = 0
R20 [s4]  = 0
R21 [s5]  = 0
R22 [s6]  = 0
R23 [s7]  = 0
R24 [t8]  = 0
R25 [t9]  = 0
R26 [k0]  = 0
R27 [k1]  = 0
R28 [gp]  = 10008000
R29 [sp]  = 7fffef1c
R30 [s8]  = 0
R31 [ra]  = 400110
```

3. Problem 3

Problem statement

Write a MIPS assembly program to swap the values of two registers using only logical instructions.

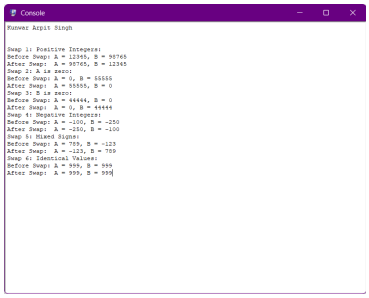
MIPS Assembly Source

```
.data
           myName: .asciiz "Kunwar␣Arpit␣Singh\n\n"
       before_msg: .asciiz "Before␣Swap:␣A␣=␣"
       after_msg: .asciiz "\nAfter␣Swap:␣␣A␣=␣"
       b_val_msg: .asciiz ",␣B␣=␣"
       header1: .asciiz "\nSwap␣1:␣Positive␣Integers:\n"
       header2: .asciiz "\nSwap␣2:␣A␣is␣zero:\n"
       header3: .asciiz "\nSwap␣3:␣B␣is␣zero:\n"
       header4: .asciiz "\nSwap␣4:␣Negative␣Integers:\n"
       header5: .asciiz "\nSwap␣5:␣Mixed␣Signs:\n"
       header6: .asciiz "\nSwap␣6:␣Identical␣Values:\n"
.text
.globl main

main:
       li $v0, 4
       la $a0, myName
       syscall

       li $s0, 12345 # A
       li $s1, 98765 # B
       la $a1, header1
       jal RunSingleSwap

       li $s0, 0
       li $s1, 55555
       la $a1, header2
       jal RunSingleSwap

       li $s0, 44444
       li $s1, 0
       la $a1, header3
       jal RunSingleSwap

       li $s0, -100
       li $s1, -250
       la $a1, header4
       jal RunSingleSwap

       li $s0, 789
       li $s1, -123
       la $a1, header5
```

```
43          jal RunSingleSwap
44
45          li $s0, 999
46          li $s1, 999
47          la $a1, header6
48          jal RunSingleSwap
49
50          li $v0, 10
51          syscall
52
53  RunSingleSwap:
54          addi $sp, $sp, -4
55              sw $ra, 0($sp)
56          li $v0, 4
57          move $a0, $a1
58          syscall
59          li $v0, 4
60          la $a0, before_msg
61          syscall
62          jal PrintState
63              and $t0, $s0, $s1
64          nor $t1, $t0, $t0
65          or $t2, $s0, $s1
66          and $s0, $t2, $t1
67          and $t0, $s0, $s1
68          nor $t1, $t0, $t0
69          or $t2, $s0, $s1
70          and $s1, $t2, $t1
71          and $t0, $s0, $s1
72          nor $t1, $t0, $t0
73          or $t2, $s0, $s1
74          and $s0, $t2, $t1
75          li $v0, 4
76          la $a0, after_msg
77          syscall
78              jal PrintState
79              lw $ra, 0($sp)
80              addi $sp, $sp, 4
81              jr $ra
82
83  PrintState:
84          li $v0, 1
85          move $a0, $s0
86          syscall
87          li $v0, 4
88          la $a0, b_val_msg
89          syscall
90          li $v0, 1
91          move $a0, $s1
92          syscall
93          jr $ra
```

Output Screenshots

Console output (Register Swap)



Register values (Register Swap)

Int Regs [16]

```
PC        = 4000bc
EPC       = 0
Cause     = 0
BadVAddr  = 0
Status    = 3000ff10

HI        = 0
LO        = 0

R0  [r0] = 0
R1  [at] = 10010000
R2  [v0] = a
R3  [v1] = 0
R4  [a0] = 3e7
R5  [a1] = 100100bc
R6  [a2] = 7fffef30
R7  [a3] = 0
R8  [t0] = 0
R9  [t1] = ffffffff
R10 [t2] = 3e7
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 3e7
R17 [s1] = 3e7
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0
R21 [s5] = 0
R22 [s6] = 0
R23 [s7] = 0
R24 [t8] = 0
R25 [t9] = 0
R26 [k0] = 0
R27 [k1] = 0
R28 [gp] = 10008000
R29 [sp] = 7fffef1c
R30 [s8] = 0
R31 [ra] = 4000b8
```

4. Problem 4

Problem statement

Write a MIPS assembly program to compute a weighted average of four 32-bit numbers (A, B, C, and D) using the formula: $(0.125A + 0.25B + 0.5C + 0.5D)$. Do not use multiplication, division, or floating-point instructions. The final value should be returned in register $a0.

MIPS Assembly Source

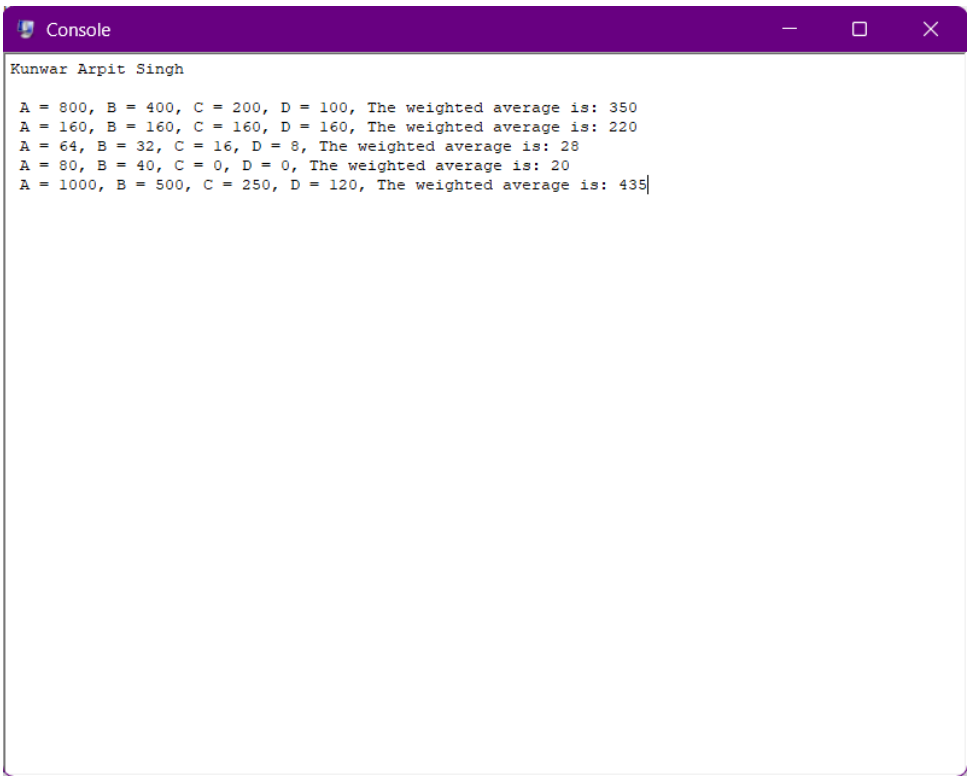
```
.data
        author_msg: .asciiz "Kunwar␣Arpit␣Singh\n"
        result_msg: .asciiz ",␣The␣weighted␣average␣is:␣"
        val_A_msg: .asciiz "\n␣A␣=␣"
        val_B_msg: .asciiz ",␣B␣=␣"
        val_C_msg: .asciiz ",␣C␣=␣"
        val_D_msg: .asciiz ",␣D␣=␣"

.text
.globl main

main:
        li $v0, 4
        la $a0, author_msg
        syscall

        li $a0, 800
        li $a1, 400
        li $a2, 200
        li $a3, 100
        jal calculate_and_print

        li $a0, 160
        li $a1, 160
        li $a2, 160
        li $a3, 160
        jal calculate_and_print

        li $a0, 64
        li $a1, 32
        li $a2, 16
        li $a3, 8
        jal calculate_and_print

        li $a0, 80
        li $a1, 40
        li $a2, 0
        li $a3, 0
        jal calculate_and_print

```

```
41          li $a0, 1000
42          li $a1, 500
43          li $a2, 250
44          li $a3, 120
45          jal calculate_and_print
46
47          li $v0, 10
48          syscall
49
50  calculate_and_print:
51
52          move $s0, $a0
53          move $s1, $a1
54          move $s2, $a2
55          move $s3, $a3
56
57
58          li $v0, 4
59          la $a0, val_A_msg
60          syscall
61
62          li $v0, 1
63          move $a0, $s0
64          syscall
65
66
67          li $v0, 4
68          la $a0, val_B_msg
69          syscall
70
71          li $v0, 1
72          move $a0, $s1
73          syscall
74
75          li $v0, 4
76          la $a0, val_C_msg
77          syscall
78
79          li $v0, 1
80          move $a0, $s2
81          syscall
82
83          li $v0, 4
84          la $a0, val_D_msg
85          syscall
86
87          li $v0, 1
88          move $a0, $s3
89          syscall
90
91          srl $t4, $s0, 3 # $t4 = A / 8
```

```
92          srl $t5, $s1, 2 # $t5 = B / 4
93          srl $t6, $s2, 1 # $t6 = C / 2
94          srl $t7, $s3, 1 # $t7 = D / 2
95
96          add $s4, $t4, $t5
97          add $s4, $s4, $t6
98          add $s4, $s4, $t7
99
100         li $v0, 4
101         la $a0, result_msg
102         syscall
103
104         li $v0, 1
105         move $a0, $s4
106         syscall
107
108         jr $ra
```

Output Screenshots

Console output (Weighted Average)



Register values (Weighted Average)

Int Regs [16]

```
PC        = 400098
EPC       = 0
Cause     = 0
BadVAddr  = 0
Status    = 3000ff10

HI        = 0
LO        = 0

R0  [r0]  = 0
R1  [at]  = 10010000
R2  [v0]  = a
R3  [v1]  = 0
R4  [a0]  = 1b3
R5  [a1]  = 1f4
R6  [a2]  = fa
R7  [a3]  = 78
R8  [t0]  = 0
R9  [t1]  = 0
R10 [t2]  = 0
R11 [t3]  = 0
R12 [t4]  = 7d
R13 [t5]  = 7d
R14 [t6]  = 7d
R15 [t7]  = 3c
R16 [s0]  = 3e8
R17 [s1]  = 1f4
R18 [s2]  = fa
R19 [s3]  = 78
R20 [s4]  = 1b3
R21 [s5]  = 0
R22 [s6]  = 0
R23 [s7]  = 0
R24 [t8]  = 0
R25 [t9]  = 0
R26 [k0]  = 0
R27 [k1]  = 0
R28 [gp]  = 10008000
R29 [sp]  = 7fffef1c
R30 [s8]  = 0
R31 [ra]  = 400094
```